

Data Structure Descriptors

Contents

- [Basic Syntax](#)
- [One-Dimensional Memory Vectors](#)
- [Two-, Three-, or Four-Dimensional Memory Vectors](#)
- [Pointers To Scalars As Destinations](#)
- [Circular Buffers](#)
- [Fabric Input Vectors](#)
- [Fabric Output Vectors](#)
- [FIFOs](#)
- [Changing DSD Properties](#)
- [Asynchronous DSD Operations](#)
- [Explicit Index Offset](#)
- [Advanced DSD Features](#)

Data Structure Descriptors (DSDs) are a compact representation of a (possibly non-contiguous) chunk of memory or a sequence of incoming or outgoing wavelets. Combined with DSD operations, DSDs enable various repeated operations to be expressed using just one hardware instruction.

All kinds of DSDs share one key property which is the **extent** or **length**. This property represents the number of repeated operations or number of iterations that the DSD represents.

Basic Syntax

DSDs are defined in CSL using the following syntax:

```
@get_dsd(dsd_type, properties);
```

Where:

- **dsd_type** is one of **mem1d_dsd**, **mem4d_dsd**, **cirdbuf_dsd**, **fabin_dsd**, or **fabout_dsd**.
- **properties** is a struct that specifies auxiliary properties of the particular DSD type.

Different DSD types require different properties.

One-Dimensional Memory Vectors

The `mem1d_dsd` type is used to encode a memory vector using a single induction variable. Memory vectors are configured using the following fields:

- `base_address`, any expression of pointer type.
- `extent`, any expression of type `u16`.
- `stride`, any expression of type `i8`. (optional, defaults to 1).
- `offset`, any expression of type `i16`. (optional, defaults to zero).
- `tensor_access`, a comptime-known tensor access expression (see [tensor_access](#)).
- `wavelet_index_offset`, a comptime-known boolean expression (optional, defaults to `false`).

tensor_access

The `tensor_access` field is a convenient grouping of properties that fully specifies a memory access pattern through an expression that is referred to as a **tensor access expression**. A tensor access expression has the following syntax:

```
|<induction-variable>| {<length>} -> <base-address>[<expr>]
```

Where:

- `induction-variable`, a single identifier that represents the loop induction variable (its iteration variable).
- `length`, a comptime-known non-negative integer expression that represents the number of times to iterate.
- `base-address`, the name of a variable of tensor type or the name of a comptime-known variable of pointer-to-tensor type.
- `<expr>`, a comma-separated list of affine expressions of the loop iteration variable and comptime-known values. There must be exactly as many affine expressions as the number of dimensions of the tensor that is specified by `base-address`.

For example:

```
const array = @zeros([10]u16);
const tenWords = @get_dsd(mem1d_dsd, .{
    .tensor_access = |i|{10} -> array[i]
});
```

In the above snippet, `tenWords` is a `mem1d_dsd` specifying accesses to `array` at indices 0 through 9.

A tensor access expression can be used to refer to odd elements of an array:

```

const array = @zeros([10]u16);
const oddElements = @get_dsd(mem1d_dsd, .{
    .tensor_access = |i|{5} -> array[2 * i + 1]
});

```

It can also be used to refer to a single element of the array:

```

const array = @zeros([10]u16);
const firstElement = @get_dsd(mem1d_dsd, .{
    .tensor_access = |i|{10} -> array[0]
});

```

Such a pattern is often useful in reduction operations.

Although `mem1d_dsd` allows only one induction variable, the underlying array can still be a multidimensional array. For instance, the following `mem1d` DSD refers to the diagonal elements of a 2D (20x20) array.

```

const array = @zeros([20,20]u16);
const diagonal = @get_dsd(mem1d_dsd, .{
    .tensor_access = |i|{20} -> array[i,i]
});

```

The tensor access expression is syntactic sugar to create an anonymous struct with the following comptime-known fields:

- `base_address`, a pointer representing the base address of the underlying access pattern.
- `offset`, an expression of type `i16` that is the access pattern's offset from `base_address`.
- `stride`, a tuple containing a single expression of type `'i8'`.
- `extent`, a tuple containing a single expression of type `'u16'`.

This also means that the type of a tensor access expression is that of the corresponding anonymous struct.

For example:

```

var A: [10]i16;
// 'access_of_A' is exactly equivalent to:
// {.base_address = &A, .offset = 42, .stride = .{2}, .extent = .{10}}
const access_of_A = |i|{10} -> A[2*i + 42];

// A 'mem1d_dsd' created through the 'tensor_access' property.
const dsd1 = @get_dsd(mem1d_dsd, {.tensor_access = access_of_A});

// A 'mem1d_dsd' created through explicitly specifying properties
// individually.
const dsd2 = @get_dsd(mem1d_dsd, {.base_address = access_of_A.base_address,
                                   .offset = access_of_A.offset,
                                   .stride = access_of_A.stride,
                                   .extent = access_of_A.extent});

```

In the example above, both ways of creating a 1D memory DSD are equivalent, which means that `dsd1` and `dsd2` are exactly the same.

As a result, it is also possible to use an anonymous struct directly as the value of the `tensor_access` field, as follows:

```

var A: [10]i16;
// The '.offset' field defaults to zero.
const access_of_A = {.base_address = &A,
                    .stride = 2,
                    .extent = 10};
const dsd = @get_dsd(mem1d_dsd, {.tensor_access = access_of_A});

```

It is not allowed to specify any DSD properties twice, which means that the `base_address`, `stride`, `extent` and `offset` fields cannot be specified in both the `tensor_access` value and as top-level properties at the same time.

Runtime `mem1d_dsd` tensor access properties

By specifying memory access properties of 1D memory DSDs individually, we are able to use runtime values for them. This is not possible through the tensor access expression since they must be compile-time-known.

For example:

```

var ptr: [*]i16;
task foo(stride: i16, len: u16) void {
    // Definition of a 'mem1d_dsd' with runtime properties.
    var dsd = @get_dsd(mem1d_dsd, {.base_address = ptr,
                                   .stride = @as(i8, stride),
                                   .extent = len});
}

```

wavelet_index_offset

The `wavelet_index_offset` field expects a comptime-known boolean value that indicates whether the `wavelet_index_offset` mode is enabled.

If the `wavelet_index_offset` mode is enabled, the address of the underlying memory buffer is incremented by the `index` specified in the DSD operation as explained in [Explicit Index Offset](#).

If a DSD with `wavelet_index_offset` enabled is used in a DSD operation, the DSD operation must provide an index field. Otherwise, the behavior of the respective DSD operation is undefined.

```
const array = @zeros([size]u16);
const memDSD = @get_dsd(mem1d_dsd, .{
    .tensor_access = |i|{10} -> array[i],
    .wavelet_index_offset = true
});

task my_task() void {
    // The addition will start at an offset specified
    // by 'my_index'.
    @add16(memDSD, memDSD, data, {.index = my_index});

    // The behavior of this builtin is undefined.
    @add16(memDSD, memDSD, data);
}
```

Two-, Three-, or Four-Dimensional Memory Vectors

The `mem4d_dsd` is a DSD type that is used to refer to multi-dimensional memory vectors, up to a maximum of four dimensions. Multi-dimensional memory vectors are configured using the following fields:

- `base_address`, a comptime-known pointer to a tensor.
- `offset`, any expression of type `i16` (optional, defaults to zero).
- `stride`, a comptime-known tuple (i.e., anonymous struct with nameless fields) of expressions of type `i16`. (optional, defaults to a tuple with values of 1 that has the same size as the `extent` tuple).
- `extent`, a comptime-known tuple (i.e., anonymous struct with nameless fields) of expressions of type `u16`.
- `tensor_access`, a tensor access expression (see [tensor_access](#)).
- `wavelet_index_offset`, a comptime-known boolean expression (optional, defaults to `false`).

tensor_access

Like in 1D memory DSDs, the `tensor_access` field is a convenient grouping of properties through a **tensor access expression**. The only difference is that in multi-dimensional memory DSDs we can have up to 4 comma-separated induction variables and length expressions and the number of induction variables and length expressions must match. For example:

```
const array = @zeros([4,3]u16);
const subset = @get_dsd(mem4d_dsd, .{
    .tensor_access = |i,j|{2,2} -> array[i, j]
});
```

The **subset** DSD will access four elements of **array** in the following order: [0, 0], [0, 1], [1, 0], [1, 1].

The following, more complicated, example shows a DSD that uses all four dimensions with non-zero offsets.

```
const array = @zeros([1,2,3,4]u16);
const subset = @get_dsd(mem4d_dsd, .{
    .tensor_access = |i,j,k,l|{1,2,1,4} -> array[i, j, 1+k, l]
});
```

Here, the **subset** DSD will access 8 elements of **array** in the following order:

```
[0,0,1,0]
[0,0,1,1]
[0,0,1,2]
[0,0,1,3]
[0,1,1,0]
[0,1,1,1]
[0,1,1,2]
[0,1,1,3]
```

mem4d DSDs can be used with single-dimensional vectors as well, like in the following, somewhat contrived, example:

```
const subset = @get_dsd(mem4d_dsd, .{
    .tensor_access = |i,j,k,l|{1,1,1,1} -> array[i + j + k + l]
});
```

In the above example, the **subset** will only access element 0 of **array**.

Like 1D memory DSDs the tensor access expression is lowered into an anonymous struct with the same fields (see [tensor_access](#)). For example:

```
var A: [20]i16;
const dsd1 = @get_dsd(mem4d_dsd, .{
    .tensor_access = |i,j|{5,5} -> A[2*i + j]
});

// Equivalent to dsd1
const dsd2 = @get_dsd(mem4d_dsd, .{
    .base_address = &A, .extent = {5, 5}, .stride = {1, -2}
});
```

The stride determines by how much the address changes when we increment the access. This must take into account the values that are reset when the access is incremented.

To understand the mapping between tensor access expression and the strides, consider the behavior of the above example. Considering each dimension in turn:

- Each time the inner dimension j is incremented, the accessed index of A increases by 1 , so stride 0 corresponding to the inner dimension is 1 .
- Each time the outer dimension i is incremented, the inner dimension j resets from its upper limit 4 to 0 . The access at $i = 0, j = 4$ is at index $2*0 + 4 = 4$, and the access at $i = 1, j = 0$ is at index $2*1 + 0 = 2$. Thus, the accessed index of A changes by $2 - 4 = -2$, so stride 1 corresponding to the outer dimension is -2 .

For a more complex example, consider:

```
var A: [4, 5]i16;
// 'access_of_A' is exactly equivalent to:
// {.base_address = &A, .offset = 2,
//  .stride = {.1,-3,-3,-23}, .extent = {.5,5,5,5}}
const access_of_A = |i, j, k, l|{5, 5, 5, 5} -> A[i + j, k + l + 2];

// A 'mem4d_dsd' created through the 'tensor_access' property.
const dsd1 = @get_dsd(mem4d_dsd, {.tensor_access = access_of_A});

// A 'mem4d_dsd' created through explicitly specifying properties
// individually.
const dsd2 = @get_dsd(mem4d_dsd, {.base_address = access_of_A.base_address,
                                   .offset = access_of_A.offset,
                                   .stride = access_of_A.stride,
                                   .extent = access_of_A.extent});
```

The array A is laid out row-major in memory. Thus, we can rewrite the expression $A[i + j, k + l + 2]$ as if it were a 1D array as $A[5 * (i + j) + k + l + 2]$. Considering each stride in turn:

- To calculate the innermost stride 0 , i.e. for l , each time l is incremented, the accessed index of A increases by 1 .
- For stride 1 , i.e. for k , each time k is incremented, the inner dimension l is reset from 4 to 0 , so the accessed index of A changes by $1 - 4 = -3$.
- For stride 2 , i.e. for j , incrementing j increases the accessed index of A by 5 , but both k and l are reset from 4 to 0 , so the accessed index of A changes by $5 - 4 - 4 = -3$.
- For stride 3 , i.e. for i , incrementing i increases the accessed index of A by 5 , but j, k , and l are reset from 4 to 0 , so the accessed index of A changes by $5 - 5*4 - 4 - 4 = -23$.

Warning

The stride values are read left to right, but the access values are read right to left. For example, for a stride of $\{1, -2\}$ and extent of $\{2, 4\}$, the innermost (fastest changing) dimension will have a stride of 1 and extent of 4 , while the outermost dimension will have a stride of -2 and extent of 2 .

wavelet_index_offset

See [wavelet_index_offset](#).

Pointers To Scalars As Destinations

Some DSD operations support pointers to scalars as destination arguments. These operations essentially behave as if the destination were a memory DSD with zero stride, whose destination is a one-element array whose data is stored at the pointer. For example:

```
const src_array = [8]f16{ 0, 1, 2, 3, 4, 5, 6, 7 };
// src_dsd will access src_array at indices, 0, 2, 4, and 6.
const src_dsd = @get_dsd(mem1d_dsd, .{
    .tensor_access = |i|{4} -> src_array[2*i]
});

const dst_array = @zeros([1]f16);
// Because dst_dsd has zero stride (`i` is never mentioned to the right of
// the arrow in the tensor access expression), the @fmovh below will first
// move 0, then 2, then 4, then 6 into dst_array[0]. Thus afterwards,
// dst_array[0] will have a value of 6.
const dst_dsd = @get_dsd(mem1d_dsd, .{
    .tensor_access = |i|{4} -> dst_array[0]
});
@fmovh(dst_array, src_dsd);
@assert(dst_array[0] == 6);

// An @fmovh operation with a pointer to scalar as its destination behaves
// similarly, so the value of dst_scalar after the following @fmovh will
// also be 6.
const dst_scalar: f16 = 0;
@fmovh(&dst_scalar, src_dsd);
@assert(dst_scalar == 6);
```

Circular Buffers

The `circbuf_dsd` type is used to implement a typical circular buffer, i.e., a contiguous one-dimensional memory buffer that will wrap around to its base address (start) once computation reaches its wraparound position (end). Circular buffer DSDs are configured using the following fields:

- `base_address`, a comptime-known pointer.
- `extent`, a comptime-known non-negative integer expression.
- `wraparound`, a comptime-known expression of type `u16`.

The `base_address` field specifies the beginning (start) of the circular buffer as well as its current position (head). Once a DSD operation reaches the end of the circular buffer then the current position (head) will reset back to the start, i.e., `base_address`.

The **extent** field specifies the number of iterations encapsulated by the DSD representation. When a **circbuf_dsd** is used as an operand to a DSD operation, the **extent** field determines how many elements are processed by that operation.

The **wraparound** field represents the number of elements from the **base_address** to the exclusive end of the circular buffer, or in other words, the address at which the wraparound will occur.

If **base_address** is a comptime-known pointer to a tensor then **wraparound** is optional and defaults to the size of the underlying tensor. If a **wraparound** is explicitly provided then it must be less than the size (in number of elements) of the underlying tensor.

For example:

```
var buffer: [10]f16;

// In this example, 'wraparound' is automatically set to 'size'.
// In this example, 'circbuf' will iterate over all the buffer
// elements twice given that the 'extent' is twice the size of
// the underlying buffer.
const circbuf1 = @get_dsd(circbuf_dsd, .{
    .base_address = &buffer, .extent = 20
});

// In this example, the wraparound will take place after 5 iterations,
// which in this case, occure after `buffer[4]` is accessed.
const circbuf2 = @get_dsd(circbuf_dsd, .{
    .base_address = &buffer, .extent = 20,
    .wraparound = 5
});

// ERROR: wraparound exceeds the size of the underlying buffer.
const circbuf3 = @get_dsd(circbuf_dsd, .{
    .base_address = &buffer, .extent = 20,
    .wraparound = 15
});

// Wraparound is required since the base address is not a pointer
// to a tensor but a pointer to a scalar.
const circbuf4 = @get_dsd(circbuf_dsd, .{
    .base_address = &buffer[offset], .extent = 20,
    .wraparound = 5
});
```

Circular buffer DSDs cannot be used as operands to DSD operations directly. They must be loaded using the **@load_to_dsr_xdsr** builtin at comptime or runtime (see [@load_to_dsr_xdsr](#)).

Fabric Input Vectors

The **fabin_dsd** DSD type is used to refer to wavelets arriving at the PE from the fabric. Fabric input vectors are configured using the following two fields.

- **input_queue**, which specifies the input queue supplying wavelets to associate with this vector

- **extent**, which specifies the number of wavelets that this vector refers to

On WSE-2, **fabric_color** can be used instead of **input_queue** to specify the color of the wavelets to associate with the vector.

For instance, the following DSD refers to 5 wavelets expected to arrive on color **trigger**.

```
const inDsd = @get_dsd(fabin_dsd, .{ .extent = 5, .fabric_color = trigger });
```

On WSE-3, it is illegal for a DSD operation to have multiple arguments that are fabric inputs.

In addition, on WSE-3, the following optional field is available:

- **priority**, an optional field that sets the priority of the microthread associated with the DSD. Possible values are **.{ .high = true }, .{ .medium = true }, .{ .low = true }**.

Fabric Output Vectors

Fabric output vectors, specified using the **fabout_dsd** type, are configured similarly to fabric input (**fabin_dsd**) vectors, with the exception that fabric output vectors may contain the following additional fields:

- **control**
- **wavelet_index_offset**

control

The **control** field expects a comptime-known boolean expression, which is used to signify control wavelets.

For instance, the following DSD refers to 1024 non-control wavelets to be sent along the color **tx**.

```
const outDsd = @get_dsd(fabout_dsd, .{ .extent = 1024, .fabric_color = tx });
```

Whereas the following DSD refers to a single control wavelet to be sent along the color **out**.

```
const dsd = @get_dsd(fabout_dsd, .{
    .extent = 1,
    .control = true,
    .fabric_color = out,
});
```

wavelet_index_offset

The **wavelet_index_offset** field expects a comptime-known boolean expression, which is used to enable the **wavelet_index_offset** mode. When this mode is enabled, the outgoing wavelets will carry a fixed index field specified by the user per-operation as explained in [Explicit Index Offset](#). Similar to the semantics of memory DSDs, if the operations that

use fabric output DSDs with `wavelet_index_offset` enabled do not specify an `index` value, then the behavior is undefined.

```
const outDSD = @get_dsd(fabout_dsd, .{
    .extent = 1,
    .fabric_color = out,
    .wavelet_index_offset = true,
});

task my_task() void {
    // The outgoing wavelets will have 'my_index' stored in
    // their high 16-bits.
    @add16(outDSD, memDSD, 42, .{.index = my_index});

    // The behavior of this builtin is undefined.
    @add16(outDSD, memDSD, 42);
}
```

FIFOs

A FIFO DSD is a kind of DSD that uses a memory region to create a First-In First-Out buffer.

To create a FIFO DSD, the `@allocate_fifo` builtin is used:

```
var fifo_buffer = @zeros([32]i16);
const fifo = @allocate_fifo(fifo_buffer);
```

The `@allocate_fifo` builtin must be associated with a `const` variable in the global scope. The argument to `@allocate_fifo` must be a global array or pointer to a global array. This array must be marked as `var` and its element type must be an ABI-compatible numeric type.

If the fifo buffer (i.e., the argument to `@allocate_fifo`) was declared without an explicit alignment requirement (by using the `align` directive (see [Variables](#)) then the compiler will force its alignment to be the minimum alignment that is required for fifos on the target architecture.

On the other hand, if the fifo buffer has been declared with an explicit alignment requirement that is less than the minimum alignment required for fifo buffers on the target architecture, an error will be raised.

Note that if the fifo buffer is declared as `extern` (see [Variables](#)) without an explicit `align` directive then a warning will be emitted indicating that proper alignment must be specified for the respective buffer definition. This warning can be suppressed by specifying an explicit alignment requirement to the `extern` fifo buffer declaration.

Allocating a FIFO consumes hardware resources for the duration of the program, as such they should be used sparingly.

The following restrictions apply when using a FIFO DSD in a DSD operation:

- The FIFO DSD operand must be compiletime-known.
- A FIFO cannot be used as an operand to the `@map` builtin.

- If a DSD operation uses more than one source operand:
 - at most one operand may be a FIFO DSD, and
 - the FIFO DSD operand must not be the first (left-most) source operand.

The `@allocate_fifo` builtin takes an optional configuration struct which can optionally contain the fields described below.

Full and Empty Actions

When a DSD operation that reads from an empty FIFO terminates, the length of the FIFO will be updated to the remaining length after the FIFO became empty. If the destination operand is a pointer to a scalar, any data popped from the FIFO during the operation will be discarded, and the value stored at the pointer will remain unchanged.

When a DSD operation that writes to a full FIFO terminates, the length of the FIFO will be updated to the remaining length after the FIFO became full.

In addition, DSD operations that read from an empty FIFO or write to a full FIFO will execute actions specified by the `.empty_action` and `.full_action` configuration struct fields, respectively. Possible actions are:

- **test_or_suspend**: If the DSD operation is synchronous, terminate the operation and return `false`. If the DSD operation is asynchronous, suspend the operation until the FIFO is no longer full or empty.
- **terminate**: Terminate the operation and return `true`.
- **suspend**: Suspend the operation until the FIFO is no longer full or empty. Not supported on WSE-2.
- **fault**: Halt execution with an unrecoverable fault. Not supported on WSE-2.

If `.empty_action` and/or `.full_action` are not specified, the corresponding action defaults to `test_or_suspend`. `.full_action` is not supported on WSE-2.

```
var fifo_buffer = @zeros([32]i16);
const fifo = @allocate_fifo(
    fifo_buffer,
    .{ .empty_action = .{ .terminate = true },
      .full_action = .{ .fault = true } } // Requires WSE-3
);
```

FIFOs are typically used with a pair of DSD operations: one operation writing elements to the FIFO and one operation reading elements from the FIFO. For example:

```
@mov16(fifo, ...); // Write to the FIFO
@mov16(..., fifo); // Read from the FIFO
```

Task Activation on Pop and Push

The `.activate_pop` configuration struct field specifies a `local_task_id` or comptime-known task name to be activated on pop from the FIFO. The `.activate_push` configuration struct field specifies a `local_task_id` or comptime-known task name to be activated on push to the FIFO. The associated task must be bound as a local task.

Note that the specified `.activate_pop` task is only activated on pop if the FIFO has previously hit a FIFO full event, and if the pop causes the FIFO to transition from having insufficient free space to having sufficient free space, where “sufficient free space” means sufficient space for the push operation that originally triggered the FIFO full event to proceed. The amount of space required depends on the operand size and SIMD width of the push operation that previously triggered the FIFO full event.

Similar rules apply in the other direction: the `.activate_push` task is only activated on push if the FIFO has previously hit a FIFO empty event, and if the push causes the FIFO to transition from having insufficient data to having sufficient data, where “sufficient data” means sufficient data in the queue for the pop operation that originally triggered the FIFO empty event to proceed.

```
task on_push() void { ... }
task on_pop() void { ... }

var fifo_buffer = @zeros([32]i16);
const fifo = @allocate_fifo(
    fifo_buffer,
    .{ .activate_pop = on_pop, .activate_push = on_push }
);
```

Changing FIFO Properties

The following builtins can be used to change the properties of a FIFO at runtime. Changing FIFO properties at comptime will be enabled in the future through the FIFO initialization builtin (i.e., `@allocate_fifo`). As was mentioned earlier, FIFOs acquire hardware resources for the duration of the program and therefore updating the properties of FIFOs happens in-place by directly accessing these hardware resources without creating new DSD values as is the case for the rest of the DSD kinds.

`@set_fifo_read_length` and `@set_fifo_write_length`

Update the length field of a FIFO that is associated with a read or write operation respectively.

Syntax

```
@set_fifo_read_length(fifo, length);
@set_fifo_write_length(fifo, length);
```

Where:

- `fifo` is a comptime-known FIFO DSD expression.
- `length` is a 16-bit unsigned integer expression that specifies the length to be applied in number of FIFO elements.

Example

```

var fifo_buffer = @zeros([32]i16);
// FIFOs are always initialized with read/write length zero.
const fifo = @allocate_fifo(fifo_buffer);
const fifo_length = 42;

// Sets the FIFO write length before a write operation.
@set_fifo_write_length(fifo, fifo_length);
@mov16(fifo, ...);

// Sets the FIFO read length before a read operation.
@set_fifo_read_length(fifo, fifo_length);
@move(..., fifo);

```

Semantics

The builtin will update the read or write length of the input FIFO in-place by modifying the underlying hardware resource directly.

Changing DSD Properties

The following builtins can be used to change DSD properties at runtime or compiletime. All of these builtins will always result in a new DSD value while the input value remains unchanged.

@set_dsd_base_addr

Create a new memory DSD value based on the input memory DSD value and base address.

Syntax

```
@set_dsd_base_addr(input_dsd, base_addr);
```

Where:

- `input_dsd` is a memory DSD expression, i.e., a DSD expression with a type that is either `mem1d_dsd` or `mem4d_dsd`.
- `base_addr` is a tensor identifier or a pointer expression whose base-type is a tensor.

Example

```

var A = @zeros([10]i16);
// Create a new DSD that is a clone of 'input_dsd' but has
// 'A' as its base-address.
var dsd1 = @set_dsd_base_addr(input_dsd, A);
// Use a pointer expression as the new base-address parameter.
var dsd2 = @set_dsd_base_addr(input_dsd, &A);

```

Semantics

The builtin returns a new memory DSD value that is a clone of the input DSD value but with the provided `base_addr` parameter as the new base address. The new base address will replace both the base address and offset (if any) of the input DSD value.

@increment_dsd_offset

Create a new memory DSD value based on the input memory DSD value, offset and tensor element type.

Syntax

```
@increment_dsd_offset(input_dsd, offset, elem_type);
```

Where:

- `input_dsd` is a memory DSD expression, i.e., a DSD expression with a type that is either `mem1d_dsd` or `mem4d_dsd`.
- `offset` is a 16-bit signed integer that specifies the offset to be applied as number of elements of `elem_type`.
- `elem_type` is a type expression that is used to convert `offset` into number of words. It must be an ABI-compatible numeric type (`u16`, `i16`, `i32`, `f16`, or `f32`).

Example

```

const A = @zeros([10, 10]f32);

// dsdA is defined as a 2x2 square that is at a diagonal
// offset within A.
const dsdA = @get_dsd(mem4d_dsd, .{
    .tensor_access = |i, j|{2, 2} -> A[i + 1, j + 1]});

// Create a new DSD that is a clone of the 'dsdA' but its
// base address is moved backwards by 10 f32 elements. In
// practice, new_dsd will have moved the 2x2 square upwards
// by one row.
var new_dsd = @increment_dsd_offset(dsdA, -10, f32);

```

Semantics

The builtin returns a new memory DSD value that is a clone of the input DSD value but with a new base address that is the result of adding the `offset` parameter to the base address of the input DSD. The `offset` parameter specifies the number of tensor elements to be added to the input DSD's base address. The builtin performs no runtime or compiletime checks for out-of-bounds accesses so the user needs to be aware of such risk.

@set_dsd_length

Create a new DSD value based on the input DSD and length.

Syntax

```
@set_dsd_length(input_dsd, length);
```

Where:

- `input_dsd` is a DSD expression with any DSD type except `mem4d_dsd`.
- `length` is a 16-bit unsigned integer that specifies the length to be applied in number of tensor elements or wavelets.

Semantics

The builtin returns a new DSD value that is a clone of the input DSD value but with the new length applied.

@set_dsd_stride

Create a new 1D memory DSD value based on the input 1D memory DSD and stride.

Syntax

```
@set_dsd_stride(input_dsd, stride);
```

Where:

- `input_dsd` is a DSD expression that must be of type `mem1d_dsd`.
- `stride` is an 8-bit signed integer that specifies the stride to be applied in number of tensor elements.

Semantics

The builtin returns a new DSD value that is a clone of the input DSD value but with the new stride applied.

Asynchronous DSD Operations

DSD operations **involving fabric operands** are allowed to happen asynchronously. This causes a new thread to start executing concurrently with any ongoing tasks and other asynchronous operations. A thread that starts executing as part of an asynchronous DSD operation is referred to as a **microthread** (see [Microthreads](#)).

A DSD operation will happen asynchronously if either of these conditions are true:

1. At least one DSD operand has a fabric DSD type, that is, **fabin_dsd** or **fabout_dsd**, and the **async** configuration is used.
2. At least one DSR operand was loaded using the **async** configuration (see [@load_to_dsr](#)).

For example:

```
// The @mov16 operation will be asynchronous.
@mov16(destination_dsd, source_dsd, {.async = true});

// The @mov16 will also be asynchronous even though ``async`` is not
// specified by the operation itself.
const source_dsr = @get_dsr(dsr_src0, 0);
@load_to_dsr(source_dsr, my_fabin_dsd, {.async = true});

// Specifying async here is not strictly necessary,
// but recommended to be explicit about the behavior of the operation.
@mov16(destination_dsd, source_dsr, {.async = true});
```

For an operation on a DSR to occur asynchronously, the DSR *must* be marked as asynchronous when a DSD is loaded to it with [@load_to_dsr](#). In this case, it is not necessary to specify **async** in the DSD operation. However, it is recommended to do so for clarity and explicitness.

All of the following configuration settings are directly applicable to DSRs when using the [@load_to_dsr](#) builtin (see [@load_to_dsr](#)).

Completion of Asynchronous DSD Operations

When an asynchronous DSD operation completes, it may optionally activate or unblock a task. The task to be activated or unblocked is specified in the last argument of the DSD operation.

For example:

```
@mov16(destination_dsd, source_dsd, {.async = true, .activate = mytask});
@mov16(destination_dsd, source_dsd, {.async = true, .unblock = mytask});
```

At most one of **activate** or **unblock** may be specified.

The **activate** field can be a **local_task_id** or a comptime-known task name. The associated task must be bound as a local task.

The **unblock** field can be a:

- WSE-2: **color**, **data_task_id**, **local_task_id**, or comptime-known task name.

- WSE-3: `input_queue`, `data_task_id`, `local_task_id`, or comptime-known task name.

As with the `.async` field, if using a DSR in an asynchronous operation, the `.activate` and `.unblock` fields *must* be specified in the `@load_to_dsr` call that loads a fabric DSD to the DSR. For example:

```
// The @mov16 will also be asynchronous, and my_task_id will
// be activated upon completion.
const source_dsr = @get_dsr(dsr_src0, 0);
@load_to_dsr(source_dsr, my_fabin_dsd,
    .{.async = true, .activate = my_task_id});

// Specifying async and activate here is not strictly necessary,
// but recommended to be explicit about the behavior of the operation.
@mov16(destination_dsd, source_dsr,
    .{.async = true, .activate = my_task_id});
```

In this case, as with `async`, it is not necessary to specify `activate` or `unblock` in the DSD operation. However, it is recommended to do so for clarity and explicitness.

Dynamic Completion Based on Control Wavelets

The completion of an asynchronous DSD operation can also be triggered by control wavelets. This capability must be explicitly enabled through the last argument of the DSD operation by specifying the `on_control` field.

For example:

```
// The asynchronous DSD operation will terminate.
@mov16(destination_dsd, source_dsd,
    .{.async = true, .on_control = .{.terminate = true}});

// The asynchronous DSD operation will terminate and task 'mytask' will be
// activated
@mov16(destination_dsd, source_dsd,
    .{.async = true, .on_control = .{.activate = mytask}});

// The asynchronous DSD operation will terminate and task 'mytask' will be
// unblocked
@mov16(destination_dsd, source_dsd,
    .{.async = true, .on_control = .{.unblock = mytask}});
```

The `terminate` action requires a boolean expression.

The `activate` action requires a `local_task_id` or task name. For `activate`, the associated task must be bound as a local task.

The `unblock` action requires a:

- WSE-2: `color`, `data_task_id`, `local_task_id`, or task name.
- WSE-3: `input_queue`, `data_task_id`, `local_task_id`, or task name.

For **unlock**, the associated task must be bound as a data or local task.

Hardware Resources and Asynchronous DSD Operations

Asynchronous operations consume two kinds of hardware resources: queues and microthreads. It is the programmer’s responsibility to ensure that concurrent asynchronous DSD operations do not share the same resource (queue or microthread).

Fabric Queues

Fabric operands involved in asynchronous DSD operations must be associated with a queue. Input/Output queues are hardware buffers where data is temporarily stored before entering or leaving the compute engine (CE) of a PE.

To specify a queue for fabric input DSDs (**fabin_dsd**), the **input_queue** attribute must be used, with a value of type **input_queue** as the queue identifier:

```
const my_fabin_dsd = @get_dsd(fabin_dsd,
                             .{...,
                             .input_queue = @get_input_queue(0), ...});
```

To specify a queue for fabric output DSDs (**fabout_dsd**), the **output_queue** attribute must be used, with a value of type **output_queue** as the queue identifier:

```
const my_fabout_dsd = @get_dsd(fabout_dsd,
                               .{...,
                               .output_queue = @get_output_queue(0), ...});
```

The hardware has a finite number of input and output queues, each with different buffering capabilities.

Queue Identifiers	WSE-2 Input Queue Length (words)	WSE-2 Output Queue Length (words)	WSE-3 Input Queue Length (words)	WSE-3 Output Queue Length (words)
0, 1	6	2	8	8
2, 3	4	6	4	8
4, 5	2	2	4	8
6, 7	2	N/A	4	8

It is the programmer’s responsibility to ensure that no two concurrent DSD operations share an output or input queue:

```

task t() void {
    @mov16(fabric_out_dsd, memory_dsd1, .{ .async = true });
    @mov16(fabric_out_dsd, memory_dsd2, .{ .async = true }); // Bad: same
                                                                // output queue
}

```

In the example, two concurrent asynchronous operations are spawned using the same `fabout_dsd` as the destination operand. Therefore, they also use the same `output_queue`, which is invalid.

It is the programmer's responsibility to ensure that there are no elements in a queue before reusing it for a different operation.

Microthreads

Asynchronous DSD operations require a hardware microthread, which is a finite resource.

A hardware microthread is identified by an integer identifier called a **microthread ID**.

On WSE-2 the microthread ID is implicitly determined by one of the input or output queues involved in the operation:

1. If a `fabout_dsd` operand is used, the microthread identifier is the same as the `output_queue` identifier.
2. Otherwise, the microthread identifier is the same as the `input_queue` identifier of the first `fabin_dsd` operand.

It is the programmer's responsibility to ensure that no two concurrent DSD operations share a microthread.

```

const fabric_out_dsd = @get_dsd(fabout_dsd,
                                .{.extent = 10,
                                   .output_queue = @get_output_queue(0)});
const fabric_in_dsd = @get_dsd(fabin_dsd,
                                .{.extent = 10,
                                   .input_queue = @get_input_queue(0)});

const mem1_dsd = @get_dsd(mem1d_dsd, ...);
const mem2_dsd = @get_dsd(mem1d_dsd, ...);

task t() void {
    @mov16(fabric_out_dsd, mem1_dsd, .{ .async = true }); // Microthread ID 0
    @mov16(mem2_dsd, fabric_in_dsd, .{ .async = true }); // Bad: same
                                                            // Microthread ID!
}

```

On WSE-3 the user has the option to explicitly specify the microthread ID of a given asynchronous DSD operation. This means that it can be different from the operands' respective queue IDs (see [Microthread IDs](#)).

Microthread Priority

The Cerebras hardware supports a priority setting for asynchronous operations. This is also called *microthread priority*.

On WSE-2, an asynchronous DSD operation with a fabric input DSD as its destination may have priority specified as follows:

```
@mov16(destination_dsd, source_dsd,  
        .{ .async = true, .priority = .{ .high = true } }));
```

Valid choices for priority are **high**, **medium**, and **low** (the default).

On WSE-3, the priority needs to be specified in the DSDs:

```
const source_dsd = @get_dsd(fabin_dsd, .{  
    .extent = 10, .input_queue = in_queue,  
    .priority = .{ .high = true }  
});  
@mov16(destination_dsd, source_dsd, .{ .async = true }));
```

or in the FIFOs

```
var fifo_buffer = @zeros([32]i16);  
const fifo = @allocate_fifo(fifo_buffer, .{ .priority = .{ .low = true } }));
```

In general, the hardware will favor the scheduling of higher-priority operations when multiple microthreads are running. Priority of the main thread, i.e., of non-async operations, may also be adjusted. By default, synchronous operations have a priority between **medium** and **low** microthreads. See [main_thread_priority](#) for information on how to adjust the main-thread priority level.

Explicit Index Offset

A DSD operation may have an **index** configuration field, which is expected to be an unsigned 16-bit integer value. If this setting is combined with the **wavelet_index_offset** property of memory and/or fabout DSDs, it will have the following semantics:

- **Memory DSDs:** the **index** value represents a word offset that is added to the base address of the underlying memory buffer.
- **Fabric Output DSDs:** the **index** value represents the index that is set to all outgoing wavelets, i.e., all outgoing wavelets will have **index** set in their high 16-bits.

For example:

```

const array = @zeros([size]u16);
const memDSD = @get_dsd(mem1d_dsd, .{
    .tensor_access = |i|{10} -> array[i],
    .wavelet_index_offset = true
});

const outDSD = @get_dsd(fabout_dsd, .{
    .extent = 1,
    .fabric_color = out,
    .wavelet_index_offset = true,
});

task my_task() void {
    // The addition will start at an offset specified
    // by 'my_index'.
    @add16(memDSD, memDSD, 42, .{.index = my_index});

    // The outgoing wavelets will have 'my_index' stored in
    // their high 16-bits.
    @add16(outDSD, memDSD, 42, .{.index = my_index});
}

```

The `index` configuration will be ignored by DSDs that do not have the `wavelet_index_offset` property enabled.

Advanced DSD Features

SIMD Mode

When using 16-bit values with fabric DSDs, it is possible to send or receive more than one value in a single wavelet using the so-called SIMD mode. The following code block shows how to use SIMD-32 mode with a fabric output DSD.

```

const out_dsd = @get_dsd(fabout_dsd, .{
    .extent = 10,
    .fabric_color = out_color,
    .simd_mode = .{ .simd_32 = true },
});

```

In `simd_32` mode, a single wavelet carries two 16-bit values. In `simd_64` mode, two wavelets must be ready, otherwise the DSD operation stalls. In `simd_32_or_64` mode, the operation proceeds (i.e. it doesn't stall) as long as at least one wavelet is ready.

On WSE-2, but not WSE-3, `simd_mode` may also be set on FIFOs:

```

const my_fifo = @allocate_fifo(
    some_buffer,
    if (@is_arch("wse2"))
        .{ .simd_mode = .{ .simd_64 = true } }
    else
        .{}
);

```

Reset a Source Operand

When the destination operand is a fabric output DSD, once the DSD operation is complete, the architecture can clear the memory vector represented by the source operand of the DSD operation. For instance, the following block of code sets the fabric output DSD properties so the memory represented by the operation's first source operand is reset to zero when the operation completes.

```

const out_dsd = @get_dsd(fabout_dsd, .{
    .extent = 10,
    .fabric_color = out_color,
    .zero = .{ .first_source = true },
});

const in_first_dsd = @get_dsd(mem1d_dsd, .{
    .tensor_access = |i|{10} -> first_source[i]
});

const in_second_dsd = @get_dsd(mem1d_dsd, .{
    .tensor_access = |i|{10} -> second_source[i]
});

// Multiply vectors and send to fabric. Reset `first_source` when complete.
@fmulh(out_dsd, in_first_dsd, out_first_dsd);

```

To reset the second source operand's memory, use `.zero = .{ .second_source = true }`. When the DSD operation has just one source operand, use `.second_source = true`. At any time, only one of `first_source` or `second_source` can be used.

Advancing Switch Positions

Fabric output DSDs can automatically advance switch positions when the last wavelet is sent. To use this feature, set the `advance_switch` field of the fabric output DSD to be true, like in the example below, which will cause the switch position for the color `out_color` to advance after all ten wavelets have been sent to the fabric.

```

const out_dsd = @get_dsd(fabout_dsd, .{
    .extent = 10,
    .advance_switch = true,
    .fabric_color = out_color,
});

```

Control Wavelet Transform

Control Wavelet Transform handles relaying control wavelets. Consider a scenario where there is a “buffering” PE which receives wavelets from the fabric and pushes them into a FIFO, using a microthread. There is also another microthread that pops data from the FIFO and sends them into the fabric. What if there is a requirement to relay control wavelets as well? By default, the approach described above cannot work since the task receives only the “index” and “data” bits of the wavelets, and the bit signifying that a wavelet is a control wavelet is outside of those bits. That means that if a control wavelet is pushed into the FIFO, the control bit is lost, so when it’s the time to pop it, it will be sent away as a regular wavelet, instead of a control wavelet.

To get around this limitation, the `control_transform` field can be used. By specifying `control_transform` to be true for the fabric input DSD, when a control wavelet is received, the two most significant bits of the index portion of the wavelet are overwritten to signify that the wavelet stored in the FIFO is a control wavelet. Then, a fabric output DSD with `control_transform` set to true can be used to reconstruct control wavelets and send them to the fabric.

```

var in_dsd = @get_dsd(fabin_dsd, .{ .fabric_color = recv_channel,
    .extent = 100,
    .input_queue = @get_input_queue(0),
    .control_transform = true });
const out_dsd = @get_dsd(fabout_dsd, .{ .extent = 100,
    .fabric_color = send_channel,
    .output_queue = @get_output_queue(1),
    .control_transform = true });

var buf = @zeros([5]u32);
const fifo = @allocate_fifo(buf);

task buffer() void {
    @mov32(fifo, in_dsd, .{ .async = true });
    @mov32(out_dsd, fifo, .{ .async = true });
}

```

Note since the two most significant bits of the index are overwritten, when `control_transform` is used, only the least significant 14 bits of the index can be utilized by the user. This property can only be used with fabric DSDs.